Desenvolvimento Web com .NET

Aula 9 – API Biblioteca
Consultar por Identificador, Editar e Deletar

Prof. Me. Lucas Teodoro dos Santos

UNIP

23 de Outubro de 2026

Roteiro da Aula

1. Consulta por ID
2. Atualização de Livro
3. Remoção de Livro
4. Movimentação de Empréstimos

UNIP
UNIVERSIDADE PAULISTA

Continuação da API Biblioteca

A API já permite consultar a lista de livros e cadastrar novos registros. Esta aula aborda as operações de consulta por identificador, atualização e remoção, além de apresentar a movimentação de empréstimos.

Para cada operação, implemente o método na `LivroService` e, em seguida, crie o endpoint correspondente na `LivrosController`.

UNIP
UNIVERSIDADE PAULISTA

Consulta de um Livro por ID

A consulta por identificador permite recuperar os dados de um **livro específico**, sem precisar consultar toda a lista.

Por exemplo, a requisição `GET /api/livros/1` solicita o livro cujo identificador é `1`.

- A controller recebe o ID informado na rota;
- A `LivroService` procura o registro por esse ID;
- Se encontrar, devolve o objeto `Livro`;
- Se não encontrar, lança uma exceção para sinalizar a ausência.

Implemente essa busca no método `ObterPorIdAsync`.

UNIP
UNIVERSIDADE PAULISTA

Implementação de ObterPorIdAsync

```
public async Task<Livro> ObterPorIdAsync(int id)
{
    try
    {
        var livro = await _context.Livros.FindAsync(id);
        if (livro is null)
        {
            throw new KeyNotFoundException($"Livro com ID {id} nao encontrado.");
        }
        return livro;
    }
    catch (InvalidOperationException ex)
    {
        throw new InvalidOperationException(
            "Nao foi possivel consultar o livro.", ex);
    }
}
```

UNIP
UNIVERSIDADE PAULISTA

Fluxo da Consulta

- Busca o livro de forma assíncrona pelo identificador recebido.
- Se ocorrer uma falha na busca, lança uma `InvalidOperationException` com uma mensagem sobre a operação.
- Se o livro não existir, lança uma `KeyNotFoundException`, interrompendo a execução da função.
- Se o livro for encontrado, devolve seus dados para a controller.
- A exceção será capturada por quem chamou a função: neste caso, a `LivrosController`, que retornará a resposta HTTP correspondente (404 ou 500).

UNIP
UNIVERSIDADE PAULISTA

Endpoint de Consulta por ID

O endpoint permite que o cliente consulte um livro pelo seu identificador por meio de uma requisição HTTP.

Na `LivrosController`, crie o método `ObterPorId` para atender à rota `GET /api/livros/{id}`.

- Recebe o identificador informado na rota;
- Chama a service para buscar o livro;
- Retorna os dados ou uma resposta HTTP de erro.

Por exemplo, `GET /api/livros/1` solicita o livro de ID `1`.

UNIP
UNIVERSIDADE PAULISTA

Implementação do Endpoint de Consulta por ID

```
[HttpGet("{id:int}")]
public async Task<ActionResult<Livro>> ObterPorId(int id)
{
    try
    {
        var livro = await _livroService.ObterPorIdAsync(id);
        return Ok(livro);
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { mensagem = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, new { mensagem = ex.Message });
    }
}
```

Fluxo do Endpoint de Consulta por ID

- Recebe uma requisição GET com um ID inteiro na rota.
- Encaminha o ID para `ObterPorIdAsync` e aguarda o resultado da busca realizada pela service.
- Se o livro for encontrado, retorna `200 OK` com os dados do livro no corpo da resposta.
- Captura a `KeyNotFoundException` lançada pela service quando o livro não existe e retorna `404 Not Found` com a mensagem que informa o ID procurado.
- Captura a `InvalidOperationException` sinalizada pela service e retorna `500 Internal Server Error`.
- Na implementação atual, a controller usa `ex.Message` no campo `mensagem` da resposta. Assim, o cliente recebe a mensagem definida na exceção lançada pela service.

Testando a Consulta por ID no Swagger

No Swagger, localize o endpoint `GET /api/livros/{id}` e informe um identificador.

- Para um ID cadastrado, a resposta será `200 OK`, com os dados do livro;
- Para um ID inexistente, a resposta será `404 Not Found`;
- Se ocorrer uma falha durante a consulta, a resposta será `500 Internal Server Error`.

Atualização de um Livro

A atualização permite alterar os dados de um livro já cadastrado. O ID identifica o registro e os dados enviados fornecem os novos valores.

Na `LivroService`, implemente `AtualizarAsync` com as seguintes etapas:

- Localizar o livro pelo ID;
- Atualizar título, autor e situação de empréstimo;
- Salvar as alterações e devolver o livro atualizado;
- Sinalizar quando o livro não existir ou a gravação falhar.

UNIP
UNIVERSIDADE PAULISTA

Implementação de AtualizarAsync

```
public async Task<Livro> AtualizarAsync(int id, Livro dados)
{
    try
    {
        var livro = await _context.Livros.FindAsync(id);
        if (livro is null)
        {
            throw new KeyNotFoundException($"Livro com ID {id} nao encontrado.");
        }
        livro.Titulo = dados.Titulo;
        livro.Autor = dados.Autor;
        livro.Emprestado = dados.Emprestado;
        await _context.SaveChangesAsync();
        return livro;
    }
    catch (DbUpdateException ex)
    {
        throw new InvalidOperationException(
            "Nao foi possivel atualizar o livro.", ex);
    }
}
```

UNIVERSIDADE PAULISTA

Fluxo da Atualização

- Busca o livro de forma assíncrona pelo ID recebido.
- Se o livro não existir, lança uma `KeyNotFoundException` com o ID procurado, interrompendo a operação.
- Substitui título, autor e situação de empréstimo pelos novos valores, mantendo o ID do registro.
- Salva as alterações no banco e devolve o livro atualizado.
- Se a gravação gerar uma `DbUpdateException`, lança uma `InvalidOperationException` com a mensagem da operação.
- As exceções serão capturadas por quem chamou a função: a `LivrosController`, que retornará a resposta HTTP.

Endpoint de Atualização de Livro

O endpoint recebe os novos dados de um livro por uma requisição `PUT /api/livros/{id}`.

Na `LivrosController`, crie o método `Atualizar`:

- Receber o ID pela rota e os dados pelo corpo da requisição;
- Chamar a service para realizar a atualização;
- Retornar o livro atualizado ou uma resposta HTTP de erro.

Por exemplo, `PUT /api/livros/1` atualiza o livro de ID `1`.

UNIP
UNIVERSIDADE PAULISTA

Implementação do Endpoint de Atualização de Livro

```
[HttpPut("{id:int}")]
public async Task<ActionResult<Livro>> Atualizar(
    int id, [FromBody] Livro dados)
{
    try
    {
        var livro = await _livroService.AtualizarAsync(id, dados);
        return Ok(livro);
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { mensagem = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, new { mensagem = ex.Message });
    }
}
```

UNIP
UNIVERSIDADE PAULISTA

Fluxo do Endpoint de Atualização de Livro

- Recebe o ID da rota e os novos dados do corpo da requisição PUT.
- Encaminha o ID e os dados para `AtualizarAsync` e aguarda a atualização realizada pela service.
- Se a atualização for concluída, retorna `200 OK` com os dados atualizados do livro.
- Captura a `KeyNotFoundException` quando o livro não existe e retorna `404 Not Found`.
- Captura a `InvalidOperationException` sinalizada pela service e retorna `500 Internal Server Error`.
- Usa `ex.Message` no campo `mensagem` da resposta: o cliente recebe a mensagem definida na exceção lançada pela service.

Testando a Atualização no Swagger

No Swagger, abra `PUT /api/livros/{id}`, informe o ID do livro e envie os novos dados no corpo da requisição:

```
{
    "titulo": "O Cortico",
    "autor": "Aluisio Azevedo",
    "emprestado": true
}
```

Se o livro existir e os dados forem válidos, a resposta será `200 OK`, com o livro atualizado.

Remoção de um Livro

A remoção permite excluir um livro do cadastro a partir de seu ID. A exclusão só é gravada no banco quando as alterações são salvas.

Na `LivroService`, implemente `RemoverAsync` com as seguintes etapas:

- Localizar o livro pelo ID;
- Marcar o registro para remoção;
- Salvar a exclusão no banco;
- Sinalizar quando o livro não existir ou a gravação falhar.

UNIP
UNIVERSIDADE PAULISTA

Implementação de RemoverAsync

```
1  public async Task RemoverAsync(int id)
2  {
3      try
4      {
5          var livro = await _context.Livros.FindAsync(id);
6          if (livro is null)
7          {
8              throw new KeyNotFoundException($"Livro com ID {id} nao encontrado.");
9          }
10         _context.Livros.Remove(livro);
11         await _context.SaveChangesAsync();
12     }
13     catch (DbUpdateException ex)
14     {
15         throw new InvalidOperationException(
16             "Nao foi possivel remover o livro.", ex);
17     }
18 }
```

Fluxo da Remoção

- Busca o livro de forma assíncrona pelo ID recebido.
- Se o livro não existir, lança uma `KeyNotFoundException` com o ID procurado, interrompendo a operação.
- Marca o livro encontrado para remoção no contexto.
- Salva as alterações, efetivando a exclusão no banco. A função termina sem devolver um objeto.
- Se a gravação gerar uma `DbUpdateException`, lança uma `InvalidOperationException` com a mensagem da operação.
- As exceções serão capturadas por quem chamou a função: a `LivrosController`, que retornará a resposta HTTP.

Endpoint de Remoção de Livro

O endpoint permite excluir um livro por uma requisição `DELETE /api/livros/{id}`.

Na `LivrosController`, crie o método `Remover`:

- Receber o ID do livro pela rota;
- Chamar a service para realizar a remoção;
- Confirmar a exclusão ou retornar uma resposta HTTP de erro.

Por exemplo, `DELETE /api/livros/1` remove o livro de ID `1`.

UNIP
UNIVERSIDADE PAULISTA

Implementação do Endpoint de Remoção de Livro

```
[HttpDelete("{id:int}")]
public async Task<IActionResult> Remover(int id)
{
    try
    {
        await _livroService.RemoverAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { mensagem = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, new { mensagem = ex.Message });
    }
}
```

UNIP
UNIVERSIDADE PAULISTA

Fluxo do Endpoint de Remoção de Livro

- Recebe uma requisição DELETE com o ID do livro na rota.
- Encaminha o ID para `RemoverAsync` e aguarda a remoção realizada pela service.
- Se a remoção for concluída, retorna `204 No Content`, sem corpo na resposta.
- Captura a `KeyNotFoundException` quando o livro não existe e retorna `404 Not Found`.
- Captura a `InvalidOperationException` sinalizada pela service e retorna `500 Internal Server Error`.
- Usa `ex.Message` no campo `mensagem` da resposta: o cliente recebe a mensagem definida na exceção lançada pela service.

UNIP
UNIVERSIDADE PAULISTA

Testando a Remoção no Swagger

No Swagger, abra `DELETE /api/livros/{id}` e informe o identificador do livro que deseja remover.

- Se o livro existir, a resposta será `204 No Content`;
- Se o ID não corresponder a um livro cadastrado, a resposta será `404 Not Found`;
- Depois da remoção, consulte novamente a lista ou o mesmo ID para verificar o resultado.

UNIP
UNIVERSIDADE PAULISTA

Movimentação de Empréstimo

Hoje, `Livro.Emprestado` informa apenas se o livro está disponível. Não registra quem o retirou nem quando isso aconteceu.

Essa mudança de situação passa a ser registrada como uma **movimentação com histórico**:

- Identificar o usuário que está retirando o livro;
- Registrar qual livro foi emprestado;
- Registrar a data e a hora da criação e da devolução;
- Manter o empréstimo associado ao usuário e ao livro.

O foco será a regra de negócio, a service e o endpoint. Os cadastros básicos seguirão o padrão já utilizado na API.

UNIP
UNIVERSIDADE PAULISTA

Entidade Usuario

Na pasta `Models`, crie a classe `Usuario` com poucos atributos:

Atributo	**Tipo em C#**	**Finalidade**
`Id`	`int`	Chave primária gerada no cadastro.
`Nome`	`string`	Nome obrigatório, até 100 caracteres.
`Email`	`string`	E-mail obrigatório, até 150 caracteres.

Use `Required`, `StringLength` e `EmailAddress` nas propriedades correspondentes, seguindo o padrão de validação de `Livro`.

Aqui, o usuário representa quem retira o livro.

Entidade Emprestimo

Na pasta `Models`, crie a classe `Emprestimo`:

Atributo	**Tipo em C#**	**Finalidade**
`Id`	`int`	Identificador do empréstimo.
`UsuarioId`	`int`	Chave estrangeira do usuário.
`LivroId`	`int`	Chave estrangeira do livro.
`DataCriacao`	`DateTime`	Data e hora da retirada.
`DataDevolucao`	`DateTime?`	Data e hora da devolução.

`DataDevolucao` começa como `null`: o empréstimo ainda está aberto. Acrescente as propriedades de navegação `Usuario` e `Livro`, dos respectivos tipos, inicializadas com `null!`.

Configuração do Banco de Dados

No `ApiDbContext`, acrescente os conjuntos das novas entidades:

```
1 public DbSet<Usuario> Usuarios => Set<Usuario>();
2 public DbSet<Emprestimo> Emprestimos => Set<Emprestimo>();
```

- O contexto passa a trabalhar com livros, usuários e empréstimos;
- Na entidade `Emprestimo`, `UsuarioId` e `LivroId` identificam os registros relacionados;
- Com as propriedades de navegação `Usuario` e `Livro`, o Entity Framework Core reconhece esses relacionamentos por convenção.

UNIP
UNIVERSIDADE PAULISTA

Seeding de Usuário

O seeding permite cadastrar um usuário inicial para testar o empréstimo, sem precisar implementar o cadastro de usuários neste momento.

No `OnModelCreating` do `ApiDbContext`, mantenha o seeding dos livros e acrescente:

```
var usuario = new Usuario
{
    Id = 1,
    Nome = "Maria Silva",
    Email = "maria@email.com"
};

modelBuilder.Entity<Usuario>().HasData(usuario);
```

No `HasData`, informe o ID do registro, mesmo que os cadastros comuns tenham o ID gerado pelo banco.

Migration dos Empréstimos

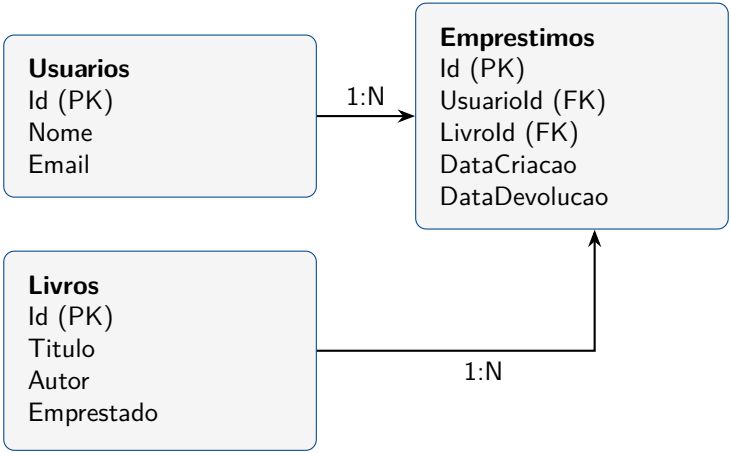
Depois de criar as entidades, atualizar o contexto e configurar o seeding do usuário, compile o projeto. No **Console do Gerenciador de Pacotes**:

```
Add-Migration AdicionarUsuariosEEmprestimos
Update-Database
```

- Uma única migration cria `Usuarios` e `Emprestimos` e inclui o usuário definido no `HasData`;
- Confira as operações `CreateTable` das novas tabelas e `InsertData` do usuário no código gerado;
- O `Update-Database` cria as tabelas e insere o usuário;
- A tabela `Livros` e seus registros existentes são mantidos;
- Use o `UsuarioId` igual a `1` para testar o empréstimo.

UNIP
UNIVERSIDADE PAULISTA

Banco de Dados com Empréstimos



Um usuário pode ter vários empréstimos. Um livro pode aparecer em vários empréstimos ao longo do tempo, mas deve ter apenas um aberto.

DTO para Realizar Empréstimo

Na pasta `DTOs`, crie `RealizarEmprestimoDto`:

```
public class RealizarEmprestimoDto
{
    [Range(1, int.MaxValue)]
    public int UsuarioId { get; set; }

    [Range(1, int.MaxValue)]
    public int LivroId { get; set; }
}
```

Use o namespace `APIBiblioteca.DTOs` e importe `System.ComponentModel.DataAnnotations` para a validação. O cliente envia apenas os identificadores necessários para a operação.

Por Que Criar a DTO?

A DTO define quais dados o cliente pode enviar para realizar o empréstimo. Ela representa a entrada da operação; a entidade representa o registro salvo.

- O cliente escolhe o usuário e o livro, enviando seus IDs;
- O ID do empréstimo será gerado no banco;
- A service define `DataCriacao` com `DateTime.UtcNow`;
- `DataDevolucao` começa como `null` e só será preenchida quando ocorrer a devolução;
- O cliente não informa os horários nem controla esses valores.

Injeção de Dependência na EmprestimoService

Na pasta `Service`, crie a classe e receba o contexto pelo construtor:

```
public class EmprestimoService
{
    private readonly ApiDbContext _context;

    public EmprestimoService(ApiDbContext context)
    {
        _context = context;
    }

    // Adicione aqui o metodo RealizarAsync.
}
```

- O ASP.NET Core fornece o `ApiDbContext` já configurado por `AddDbContext`, como no cadastro de livros;
- O construtor guarda essa referência em `_context`;
- A service usa o contexto para consultar e salvar os dados.

Implementação do Empréstimo: Validações

O método começa validando o usuário, o livro e sua disponibilidade:

```
public async Task<Emprestimo> RealizarAsync(RealizarEmprestimoDto dados)
{
    var usuario = await _context.Usuarios.FindAsync(dados.UsuarioId);
    if (usuario is null)
    {
        throw new KeyNotFoundException(
            $"Usuario com ID {dados.UsuarioId} nao encontrado.");
    }
    var livro = await _context.Livros.FindAsync(dados.LivroId);
    if (livro is null)
    {
        throw new KeyNotFoundException(
            $"Livro com ID {dados.LivroId} nao encontrado.");
    }
    if (livro.Emprestado)
    {
        throw new ArgumentException("O livro ja esta emprestado.");
    }
    // Continuacao no proximo slide.
```

Implementação do Empréstimo: Gravação

Continuação do mesmo método `RealizarAsync`:

```
    var emprestimo = new Emprestimo
    {
        UsuarioId = usuario.Id,
        LivroId = livro.Id,
        DataCriacao = DateTime.UtcNow,
        DataDevolucao = null
    };
    _context.Emprestimos.Add(emprestimo);
    livro.Emprestado = true;
    try
    {
        await _context.SaveChangesAsync();
        return emprestimo;
    }
    catch (DbUpdateException ex)
    {
        throw new InvalidOperationException(
            "Nao foi possivel realizar o emprestimo.", ex);
    }
}
```

UNIVERSIDADE PAULISTA

Fluxo da Movimentação de Empréstimo

- Confirma a existência do usuário e do livro;
- Impede a retirada se o livro já estiver emprestado;
- Cria o empréstimo com o usuário, o livro e o horário da retirada;
- Marca o livro como emprestado;
- Salva o novo empréstimo e a alteração do livro;
- Devolve o empréstimo criado ou lança uma exceção que será capturada pela controller.

Uma única operação do negócio envolve duas entidades: registrar o empréstimo e atualizar a situação do livro.

Endpoint de Realização de Empréstimo

Disponibilize `POST /api/emprestimo`, recebendo `RealizarEmprestimoDto` no corpo da requisição.

- A `EmprestimoController` recebe a requisição;
- A `EmprestimoService` realiza a movimentação;
- A controller devolve o registro criado ou uma resposta de erro.

Primeiro, configure a injeção de dependência da service na controller.

Injeção de Dependência na EmprestimoController

```
1  [ApiController]
2  [Route("api/[controller]")]
3  public class EmprestimoController : ControllerBase
4  {
5      private readonly EmprestimoService _emprestimoService;
6
7      public EmprestimoController(EmprestimoService emprestimoService)
8      {
9          _emprestimoService = emprestimoService;
10     }
11
12     // Adicione aqui o endpoint Realizar.
13 }
```

No `Program.cs`, antes de `builder.Build()`, registre:

```
1  builder.Services.AddScoped<EmprestimoService>();
```

O registro permite ao ASP.NET Core fornecer a service ao construtor da controller. `AddScoped` reutiliza a instância dentro da mesma requisição; a service recebe o contexto também por injeção.

UNIP UNIVERSIDADE PAULISTA

Implementação do Endpoint de Empréstimo

```
[HttpPost]
public async Task<ActionResult<Emprestimo>> Realizar(
    [FromBody] RealizarEmprestimoDto dados)
{
    try
    {
        var emprestimo = await _emprestimoService.RealizarAsync(dados);
        return StatusCode(201, emprestimo);
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { mensagem = ex.Message });
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { mensagem = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, new { mensagem = ex.Message });
    }
}
```

UNIVERSIDADE PAULISTA

Fluxo do Endpoint de Empréstimo

- Recebe os IDs do usuário e do livro e chama a service;
- Se o empréstimo for concluído, retorna `201 Created` com o registro criado, incluindo ID e horário;
- Retorna `404 Not Found` se o usuário ou o livro não existir;
- Retorna `400 Bad Request` se o livro já estiver emprestado. IDs fora da faixa válida também geram 400, pela validação da DTO;
- Retorna `500 Internal Server Error` para a falha de gravação sinalizada pela service;
- Nos blocos `catch`, usa `ex.Message`: a mensagem devolvida ao cliente foi definida pela exceção lançada na service.

Testando o Empréstimo no Swagger

Use o usuário criado pelo seeding e tenha um livro disponível no banco.

Em `POST /api/emprestimo`, envie IDs que existam no seu banco:

```
{
    "usuarioId": 1,
    "livroId": 1
}
```

- Confira o 201, a data de criação e a devolução nula;
- Consulte o livro: `emprestado` deve ser `true`;
- Repita a retirada: deve retornar 400;
- Teste um ID inexistente: deve retornar 404.

Uma Base para o Desenvolvimento do PIM

O exemplo de empréstimo apresentado nesta aula serve como **base para o desenvolvimento das movimentações do PIM**. As entidades e as regras podem ser adaptadas ao problema de cada projeto.

Projeto	**Movimentação**
Estoque	Registrar entrada ou saída e atualizar a quantidade.
Vendas	Criar a venda, registrar os itens e atualizar o estoque.
Agendamentos	Vincular cliente, serviço e horário disponível.
Equipamentos	Registrar retirada, responsável e devolução.

A implementação segue o mesmo raciocínio: **definição da DTO, validação das regras na service, relacionamento entre entidades e gravação das alterações**, seguida da criação do endpoint e dos testes da movimentação.

UNIP
UNIVERSIDADE PAULISTA

Resumo da Aula

A aula apresenta as operações da API Biblioteca e uma movimentação que relaciona diferentes entidades.

- **Consulta por ID:** localizar um livro específico e informar quando ele não for encontrado;
- **Atualização de Livro:** buscar o registro, alterar seus dados e salvar as mudanças;
- **Remoção de Livro:** localizar o livro e efetivar sua exclusão no banco;
- **Movimentação de Empréstimos:** validar o usuário e a disponibilidade do livro, registrar a retirada e atualizar a situação do livro.

UNIP
UNIVERSIDADE PAULISTA

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA